Giải Thích Code Giải Quyết Bài Toán N-Queens

Bài toán N-Queens yêu cầu đặt N quân hậu trên bàn cờ N×N sao cho không có quân hậu nào có thể tấn công quân khác. Code này giải quyết bài toán bằng thuật toán di truyền (genetic algorithm). Dưới đây là giải thích chi tiết về mỗi phần.

Import Thư Viện

python Copy

```python
import random
```

- Thư viện `random` được sử dụng để tạo số ngẫu nhiên, cần thiết cho việc khởi tạo quần thể, quá trình chọn lọc, lai ghép và đột biến trong thuật toán di truyền.

Lớp Queen

python Copy

```python
class Queen:
  def __init__(self, row, col):
    self.row = row
    self.col = col
```

- Định nghĩa lớp `Queen` đại diện cho một quân hậu trên bàn cờ.
- `self.row`: Hàng của quân hậu (0-indexed).
- `self.col`: Cột của quân hậu (0-indexed).
- Mỗi quân hậu được xác định bởi vị trí (row, col) của nó.

Lớp Board

python Copy

```python
class Board:
  def __init__(self, size):
    self.size = size
    self.queens = [Queen(i, random.randint(0, size - 1)) for i in range(size)]
```

- Lớp `Board` đại diện cho một bàn cờ có `size x size` ô.
- `self.size`: Kích thước của bàn cờ (số hàng và số cột).
- `self.queens`: Danh sách các quân hậu trên bàn cờ.
- Trong constructor, mỗi quân hậu được đặt ở một hàng khác nhau (từ 0 đến size-1) và cột ngẫu nhiên.

Phương thức `fitness()`

python Copy

```python
def fitness(self):
  # Calculate the number of non-attacking pairs of queens
  # Higher fitness is better (fewer conflicts)
  attacks = 0
  for i in range(self.size):
    for j in range(i + 1, self.size):
        qi = self.queens[i]
        qj = self.queens[j]

        # Check if queens attack each other (same column or diagonal)
        if qi.col == qj.col:  # Same column
            attacks += 1
        elif abs(qi.row - qj.row) == abs(qi.col - qj.col):  # Diagonal
            attacks += 1

  # Maximum possible attacks for n queens is n*(n-1)/2
  max_attacks = (self.size * (self.size - 1)) // 2

  # Fitness is inversely proportional to number of attacks
  # Perfect solution has fitness = max_attacks (no attacks)
  return max_attacks - attacks
```

- Phương thức `fitness()` tính độ thích nghi (fitness score) của một cấu hình bàn cờ.
- `attacks`: Số cặp quân hậu có thể tấn công nhau.
- Vòng lặp kiểm tra từng cặp quân hậu để đếm số cặp có thể tấn công nhau:
  - Nếu 2 quân hậu ở cùng cột (`qi.col == qj.col`), chúng có thể tấn công nhau.
  - Nếu 2 quân hậu ở cùng đường chéo (`abs(qi.row - qj.row) == abs(qi.col - qj.col)`), chúng có thể tấn công nhau.
- `max_attacks`: Số cặp quân hậu tối đa có thể có = n*(n-1)/2 (công thức tổ hợp chập 2 của n).
- Giá trị `fitness` = `max_attacks - attacks`: Đo lường số cặp quân hậu KHÔNG tấn công nhau.
- Fitness càng cao càng tốt. Giải pháp tối ưu có fitness = max_attacks (không có cặp nào tấn công nhau).

Phương thức `crossover()`

python Copy

```python
def crossover(self, other):
  # Two-point crossover
  n = self.size
  c1 = random.randint(1, n-2)  # First crossover point
  c2 = random.randint(c1+1, n-1)  # Second crossover point

  child = Board(self.size)

  # Take queens from first parent up to first crossover point
  for i in range(0, c1):
    child.queens[i].col = self.queens[i].col

  # Take queens from second parent between first and second crossover points
  for i in range(c1, c2):
    child.queens[i].col = other.queens[i].col

  # Take queens from first parent after second crossover point
  for i in range(c2, n):
    child.queens[i].col = self.queens[i].col

  return child
```

- Thực hiện lai ghép hai điểm (two-point crossover) giữa hai cá thể.
- `c1` và `c2`: Hai điểm cắt ngẫu nhiên, với `c1` < `c2`.
- Tạo một cá thể con mới (`child`).
- Sao chép cột của các quân hậu từ:
  - Cha thứ nhất (self) cho các hàng từ 0 đến c1-1
  - Cha thứ hai (other) cho các hàng từ c1 đến c2-1
  - Cha thứ nhất (self) cho các hàng từ c2 đến n-1
- Trả về cá thể con sau khi lai ghép.

Phương thức `mutate()`

python Copy

```python
def mutate(self, mutation_rate):
  # Randomly change column positions based on mutation rate
  for i in range(self.size):
    if random.random() < mutation_rate:
        # Move queen to a random column in the same row
        self.queens[i].col = random.randint(0, self.size - 1)
  return self
```

- Thực hiện đột biến ngẫu nhiên trên một cá thể.
- `mutation_rate`: Xác suất đột biến cho mỗi quân hậu.
- Với mỗi quân hậu, nếu một số ngẫu nhiên < `mutation_rate`, quân hậu sẽ di chuyển đến một cột ngẫu nhiên trong cùng hàng.
- Trả về cá thể sau khi đột biến.

Các Hằng Số và Biến Toàn Cục

python Copy

```python
board_size = 16
population_size = 256
mutation_rate = 0.2
```

- `board_size`: Kích thước bàn cờ (16x16).
- `population_size`: Số lượng cá thể trong quần thể (256).
- `mutation_rate`: Tỷ lệ đột biến (0.2 = 20%).

Hàm `random_selection()`

python Copy

```python
def random_selection(population, fitness_fn):
  # Tournament selection
  tournament_size = 5
  selected = random.sample(population, tournament_size)
  selected.sort(key=fitness_fn, reverse=True)
  return selected[0]
```

- Thực hiện chọn lọc giải đấu (tournament selection).
- `tournament_size`: Số lượng cá thể tham gia giải đấu (5).

- `random.sample(population, tournament_size)`: Chọn ngẫu nhiên `tournament_size` cá thể từ quần thể.
- Sắp xếp các cá thể đã chọn theo giá trị fitness giảm dần.
- Trả về cá thể có fitness cao nhất trong giải đấu.

Hàm `reproduce()`

python Copy

```python
def reproduce(x, y):
  return x.crossover(y)
```

- Thực hiện sinh sản giữa hai cá thể.
- Gọi phương thức `crossover()` của cá thể `x` với cá thể `y`.
- Trả về cá thể con sau khi lai ghép.

Hàm `genetic_algorithm()`

python Copy

```python
def genetic_algorithm(board_size, population_size, mutation_rate, generations):
  # Initial population
  population = [Board(board_size) for _ in range(population_size)]
  _ = 0
  for generation in range(generations):
    # Get current best solution
    population.sort(key=lambda x: x.fitness(), reverse=True)
    best_board = population[0]
    best_fitness = best_board.fitness()

    # Print progress
    max_fitness = (board_size * (board_size - 1)) // 2

    if _ < best_fitness:
      _ = best_fitness
      print(f"Generation {generation}: Best fitness = {best_fitness}/{max_fitness}")

    # Check if optimal solution found
    if best_fitness == max_fitness:
      print("Found optimal solution!")
      return best_board

    # Create new population
    new_population = []

    for i in range(population_size):
      # Select parents
      x = random_selection(population, lambda board: board.fitness())
      y = random_selection(population, lambda board: board.fitness())

      # Reproduce
      child = reproduce(x, y)

      # Mutate
      if random.random() < mutation_rate:
        child.mutate(mutation_rate)

      new_population.append(child)

    population = new_population

  # Return best solution after all generations
  population.sort(key=lambda x: x.fitness(), reverse=True)
  return population[0]
```

- Thực hiện thuật toán di truyền.
- `population`: Khởi tạo quần thể ban đầu với `population_size` cá thể ngẫu nhiên.
- `_`: Biến lưu giá trị fitness tốt nhất đã tìm thấy (để chỉ in ra khi có cải tiến).
- Vòng lặp chính: chạy qua mỗi thế hệ:
  - Sắp xếp quần thể theo giá trị fitness giảm dần.
  - `best_board`: Cá thể có fitness cao nhất trong quần thể hiện tại.
  - `best_fitness`: Giá trị fitness của cá thể tốt nhất.
  - `max_fitness`: Giá trị fitness tối đa có thể đạt được.
  - In tiến trình nếu tìm thấy giải pháp tốt hơn.
  - Nếu tìm thấy giải pháp tối ưu (best_fitness = max_fitness), trả về cá thể đó.
  - Tạo quần thể mới:
    - Với mỗi cá thể trong quần thể mới:
      - Chọn hai cha mẹ bằng hàm `random_selection()`.
      - Sinh sản bằng hàm `reproduce()`.
      - Đột biến nếu số ngẫu nhiên < mutation_rate.
      - Thêm cá thể con vào quần thể mới.
  - Thay thế quần thể cũ bằng quần thể mới.
- Trả về cá thể tốt nhất sau khi hoàn thành tất cả các thế hệ.

Chạy Thuật Toán và In Kết Quả

python Copy

```python
solution = genetic_algorithm(board_size, population_size, mutation_rate, generations = 100)
```

- Chạy thuật toán di truyền với 100 thế hệ và lưu giải pháp tốt nhất vào `solution`.

python Copy

```python
print("Final solution:")
board = [["." for _ in range(board_size)] for _ in range(board_size)]
for queen in solution.queens:
  board[queen.row][queen.col] = "Q"

for row in board:
  print(" ".join(row))

# Print queen positions
print("\nQueen positions (columns):")
print([queen.col + 1 for queen in solution.queens])
```

- In ra giải pháp cuối cùng:
  - Hiển thị bàn cờ với "Q" đại diện cho quân hậu và "." đại diện cho ô trống.
  - In ra vị trí cột của mỗi quân hậu (được cộng thêm 1 để chuyển từ 0-indexed sang 1-indexed).

Ý Nghĩa Output

Kết quả chạy thuật toán cho thấy:

- Thuật toán đã tìm thấy giải pháp tối ưu ở thế hệ thứ 44.
- Giải pháp có fitness = 120/120, nghĩa là không có cặp quân hậu nào tấn công nhau.
- Vị trí của 16 quân hậu được hiển thị trên bàn cờ và dưới dạng danh sách vị trí cột.